



ITCS442

Relational Database Design

Project Phase 4: Implementing a Database

Group 5: Restaurant Inventory Management System

Submitted by

6688072 Kemjira Nugboon

6688099 Nuttanan Reamprasert

6688129 Patsatraporn Thongdeesakul

Presented to

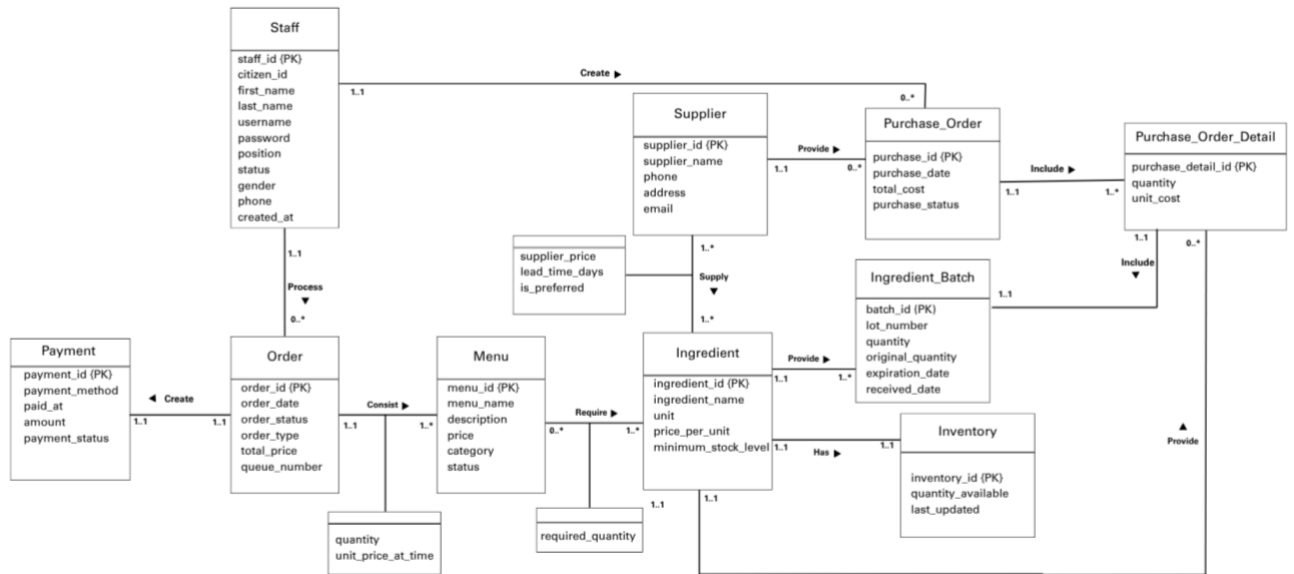
Asst.Prof. Dr. Charnyote Pluempitiwiriyawej

Relational Schema



https://www.canva.com/design/DAHCOhrZJBg/9w6foSnF9FGcTr8VYlcpDA/view?utm_content=DAHCOhrZJBg&utm_campaign=designshare&utm_medium=link2&utm_source=uniquelinks&utm_id=h70c4c9793f

ER Diagram



https://www.canva.com/design/DAHB5HgXYZQ/W8SIX0x9J-WS5ZSofRo3WQ/edit?utm_content=DAHB5HgXYZQ&utm_campaign=designshare&utm_medium=link&utm_source=sharebutton

Database Implementation

3.1 Database Creation

The database was implemented using SQL on Microsoft SQL Server based on the normalized relational schema designed in the previous phase. The system consists of multiple tables representing key entities in a restaurant inventory and ordering system, including Staff, Orders, Menu, Payment, Supplier, Ingredient, Inventory, and Purchase_Order.

The `CREATE TABLE` statements were used to define each table along with appropriate data types such as `INT`, `VARCHAR`, `DECIMAL`, `DATE`, and `DATETIME`. Each table was carefully structured to reflect real-world entities and their relationships.

Several constraints were applied to ensure data integrity and consistency:

- **Primary Keys** were defined for all tables to uniquely identify each record (e.g., `staff_id` in Staff, `order_id` in Orders, and `ingredient_id` in Ingredient).
- **Foreign Keys** were used to establish relationships between tables. For example:
 - `staff_id` in Orders references the Staff table
 - `order_id` in Payment references the Orders table
 - `menu_id` and `ingredient_id` are linked through Menu_Ingredient
 - `supplier_id` and `ingredient_id` are linked through Supplier_Ingredient
 - `purchase_id` in Purchase_Order_Detail references Purchase_Order
- **Composite Primary Keys** were used in junction tables such as Order_Menu, Menu_Ingredient, and Supplier_Ingredient to resolve many-to-many relationships.
- Additional constraints such as `NOT NULL` (implicitly applied to primary keys) help prevent missing critical data.

This design ensures that the database is normalized, reduces redundancy, and maintains strong referential integrity across all related tables.

3.2 Data Loading

The dataset was generated to simulate realistic operations of a restaurant, including ordering, payment processing, inventory management, and supplier transactions.

The data includes multiple categories:

- Operational data (Orders, Order_Menu, Payment)
- Master data (Staff, Supplier, Ingredient, Menu)
- Inventory data (Inventory, Ingredient_Batch)
- Procurement data (Purchase_Order, Purchase_Order_Detail)

The number of records in each table was designed to meet the project requirements and reflect real-world usage:

- Staff: approximately 40 records
- Supplier: approximately 15 records

- Ingredient: approximately 125 records
- Inventory: approximately 120 records
- Ingredient_Batch: approximately 400 records
- Menu: approximately 75 records
- Menu_ingredient: approximately 320 records
- Orders: approximately 25530 records
- Payment: approximately 25530 records
- Order_Menu: approximately 132505 records
- Purchase_Order: approximately 350 records
- Purchase_Order_Detail: approximately 1750 records
- Supplier_Ingredient: approximately 800 records

Transactional tables such as Orders, Payment, and Order_Menu contain larger datasets to simulate high-frequency operations in a restaurant environment. Data patterns were also applied, such as varying order types (Dine-in, Takeaway, Delivery), different order statuses, and realistic timestamps.

NOTE:

All CSV files must be stored in C:\csv before executing this script. This script uses BULK INSERT to load data from local files. If the files are not placed in this directory, the file path in the script must be modified accordingly to match the actual location of the CSV files before running the script.

3.3 Record Count

To verify that all data was successfully inserted into the database, SQL queries were used to count the number of records in each table. The `COUNT(*)` function was applied to ensure completeness and correctness of the data loading process.

Example SQL queries are shown below:

```
SELECT COUNT(*) AS Staff_Count FROM Staff;
SELECT COUNT(*) AS Orders_Count FROM Orders;
```

These queries confirm that each table contains the expected number of records and help ensure that the database has been correctly populated according to the design.

	table_name	total
1	ingredient	125
2	ingredient_batch	400
3	inventory	120
4	menu	75
5	menu_ingredient	320
6	orders	25530
7	order_menu	132505
8	payment	25530
9	purchase_order	350
10	purchase_order_detail	1750
11	supplier	15
12	supplier_ingredient	800
13	staff	40

Discussion

1. Difficulty

- One of the main difficulties in this project was ensuring data consistency across multiple related tables. Since the system consists of several interconnected tables such as Orders, Payment, Menu, and Inventory, careful design and validation of relationships were required.

Example: If a record in the Payment table contains an order_id that does not exist in the Orders table, the data insertion will fail.

- Managing **foreign keys (FK)** was particularly challenging, as all referenced values must correspond to existing records.
- Another difficulty was designing a normalized database schema to reduce redundancy while maintaining clarity and ease of implementation.
- In addition, generating a large dataset was time-consuming, especially when ensuring that the data is both realistic and consistent across all tables.

2. Challenges

- A key challenge was maintaining **referential integrity** across all tables to ensure that relationships remain valid.

Example: Each Payment record must be linked to an existing Order, and each Order must be correctly associated with Menu items through the Order_Menu table.

- Another challenge was creating realistic data patterns, such as simulating peak hours, different order types, and various statuses.
- During data generation, **duplicate data** occasionally occurred, particularly when generating large volumes of data.
Example: Multiple orders could have very similar patterns, such as the same menu items, close timestamps, and identical statuses.
- Balancing the distribution of statuses (e.g., Completed, Pending, Cancelled) was also challenging, as unrealistic distributions could reduce the quality of the dataset.

3. Solutions

- To ensure data consistency, validation rules were applied during data generation across all tables.

Example: The system checks that order_id in the Payment table exists in the Orders table and that the payment amount matches the total value of the corresponding order.

- Foreign key values were controlled within valid ranges and verified before insertion. To reduce duplicate data, the data generation process was enhanced with additional constraints and controlled randomness.

Example: Random values were generated within defined ranges to avoid excessive repetition of similar records.

- Realistic data patterns were introduced, such as higher order volumes during lunch and dinner periods. The database schema was designed based on normalization principles to reduce redundancy and improve data structure.

4. Limitation

- The system does not support advanced real-world features such as automated payment processing or real-time operations.

Example: There is no integration with a real payment gateway, so transactions cannot be verified in real time.

- The dataset is static and manually generated, which limits its ability to reflect real user behavior. There is no automation implemented, such as triggers or stored procedures.

Example: There is no trigger to automatically calculate the total order value, requiring manual calculation during data generation.

- The system is primarily developed for academic purposes and may not fully represent the complexity of a real-world database system.